

PHARMAGARDE CM

Cahier des charges fonctionnel – Version 2.0 (méthodologie alignée POSTrack)

Plateforme de recherche, vérification et confirmation des pharmacies
de garde au Cameroun

Statut : version de référence pour la poursuite du développement
Date : septembre 2026

1.2 Historique et statut du document

Version	Évolution	Statut
1.0	Cahier des charges initial : présentation, problématique, objectifs, rôles, fonctionnalités, architecture indicative (FastAPI/Python).	Archivée
2.0	Restructuration au format méthodologique POSTrack : historique versionné, navigation cible détaillée, matrice de rôles, modèle de données formalisé, API REST orientée ressource, exigences non fonctionnelles chiffrées, plan de recette. Backend remplacé par Node.js .	Version courante

Cette révision ne remet pas en cause le périmètre validé en v1.0 (recherche, fiabilité, confirmation de garde, signalement, back-office). Elle formalise la façon dont ce périmètre est structuré, développé et recetté.

1.3 Préambule

PharmaGarde CM est une plateforme numérique destinée à aider les utilisateurs à trouver rapidement les pharmacies de garde dans les villes du Cameroun, à vérifier leur statut et à obtenir un itinéraire pour s'y rendre. La plateforme privilégie la **fiabilité de l'information** sur la simple exhaustivité d'une liste.

Vision. Devenir une référence numérique pour la recherche de pharmacies de garde au Cameroun, avec une information actualisée, vérifiable et accessible.

Le présent document constitue la référence fonctionnelle et technique pour la poursuite du développement. Les livrables suivants sont considérés comme structurants et ne doivent pas être remis en cause sans arbitrage documenté : le modèle de données (User, Pharmacy, DutySchedule, Confirmation, Report, AuditLog), l'authentification par rôles, et le principe de traçabilité systématique de toute donnée critique (source, date, acteur).

1.4 Contexte, objectifs et principes directeurs

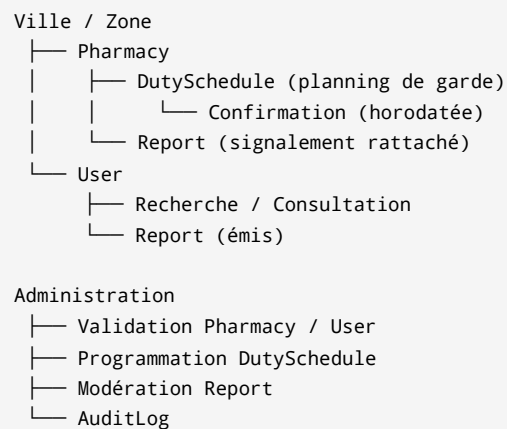
Les horaires et pharmacies de garde sont parfois difficiles à trouver rapidement ; une liste ancienne ou non confirmée peut conduire l'utilisateur vers une pharmacie fermée. Les utilisateurs ont besoin de localisation, téléphone et itinéraire au même endroit. Les pharmacies ont besoin d'un moyen simple de gérer leurs informations et de confirmer leurs gardes.

Objectif	Résultat attendu
Rechercher rapidement	Un parcours court : ville/position → liste triée par proximité → fiche pharmacie.
Garantir la fiabilité	Chaque donnée critique porte une source, une date et un statut de vérification.

Objectif	Résultat attendu
Distinguer déclaration et preuve	La confirmation de garde par la pharmacie est un signal de confiance horodaté, jamais une certitude absolue.
Outiller la pharmacie	Espace professionnel dédié à la gestion du planning et à la confirmation de garde.
Superviser la qualité	Back-office de validation, de modération des signalements et d'audit.
Préparer l'expansion	Architecture pensée pour une ville pilote, répliquable à d'autres villes sans refonte.

1.5 Périmètre fonctionnel

1.5.1 Chaîne métier



Le rattachement d'une `DutySchedule` à une `Pharmacy`, et d'une `Confirmation` à une `DutySchedule`, est obligatoire. La ville sélectionnée par l'utilisateur (ou détectée par géolocalisation) établit le

périmètre géographique de la recherche ; elle n'est pas un contrôle d'accès (l'application reste publique en lecture pour les visiteurs), à la différence du contexte Partenaire de POSTrack qui, lui, conditionne l'accès aux données.

1.5.2 Navigation cible

Parcours utilisateur (public)

Étape	Écran ou service	Contrôle
1	Accueil	Détection de position (opt-in) ou choix manuel de ville.
2	Recherche	Résolution des gardes valides pour la date/heure courante.
3	Liste / Carte	Tri par proximité et niveau de fiabilité.
4	Fiche pharmacie	Affichage source, dernière confirmation, statut.
5	Action	Appeler / Itinéraire / Signaler.

Parcours pharmacie (authentifié)

```
Login
↓
Espace Pharmacie (Dashboard)
├─ Profil & informations
├─ Planning de garde
├─ Confirmation de garde en cours
├─ Signalements reçus
└─ Historique des confirmations
```

Parcours administration (authentifié)

```
Login Admin/Super Admin
↓
```

Back-office

- └ Validation Pharmacies & Comptes
- └ Villes & Zones
- └ Programmation / validation des gardes
- └ Modération des signalements
- └ Statistiques
- └ Journal d'audit

1.5.3 Rôles applicatifs

Rôle	Périmètre	Responsabilités principales
Visiteur / Utilisateur	Public, lecture seule + signalement	Rechercher, filtrer, localiser, appeler, obtenir l'itinéraire, signaler une anomalie.
Pharmacie	Son propre profil et planning	Gérer profil et horaires, gérer le planning de garde, confirmer sa présence, traiter les alertes qui la concernent.
Administrateur	Ville(s) ou zone(s) assignée(s)	Valider pharmacies et comptes, gérer gardes, modérer signalements, consulter statistiques.
Super administrateur	Plateforme entière	Gestion complète : rôles, paramètres, sécurité, audit, arbitrage sur toutes les villes.

Le contrôle d'accès est appliqué à trois niveaux, comme dans POSTrack : la route API, le service métier et la donnée elle-même (une **Pharmacy** ne peut être modifiée que par son titulaire ou un administrateur de sa zone).

1.5.4 Modules par espace

Espace	Modules
Application utilisateur	Choix de ville / détection de position, carte, liste triée par distance, filtres (ville, quartier, statut, confirmation), fiche pharmacie, appel, itinéraire, signalement, favoris (version ultérieure).
Espace pharmacie	Compte professionnel, informations pharmacie, calendrier de garde, confirmation de garde, notifications, réponse aux signalements, historique des confirmations.
Back-office	Tableau de bord, gestion pharmacies/comptes, gestion villes/zones, programmation/validation des gardes, modération des signalements, journal d'audit, statistiques.

1.6 Workflows métier

1.6.1 Recherche et consultation

L'utilisateur choisit une ville ou autorise la géolocalisation. Le système recherche les gardes valides pour la date et l'heure courantes, classe les résultats par proximité et fiabilité, puis affiche la fiche pharmacie avec planning, statut de confirmation et date de dernière mise à jour.

1.6.2 Confirmation de garde

La confirmation est un mécanisme de confiance, pas une preuve absolue d'ouverture : elle doit toujours être accompagnée d'un horodatage et d'un statut explicite.

Planning valide → notification à la pharmacie → ouverture de l'espace pharmacie → action *Confirmer la garde* → enregistrement serveur de la date/heure → publication du statut → mise à jour du compteur de fraîcheur → rappel automatique si absence de confirmation à l'approche de l'échéance.

L'absence de confirmation **ne signifie pas automatiquement** que la pharmacie est fermée ; elle dégrade seulement le niveau de fiabilité affiché.

1.6.3 Signalement

Signalement utilisateur → création du ticket → classification → contrôle automatique → notification à la pharmacie → réponse pharmacie ou vérification admin → décision → mise à jour des données → clôture et journalisation.

Statut	Signification
Nouveau	Signalement reçu, non encore traité.
En vérification	Vérification en cours.
Confirmé	Le problème est considéré comme réel.
Rejeté	Le signalement n'est pas suffisamment fondé.
Résolu	Les données ont été corrigées.

1.6.4 Gestion d'une pharmacie

Demande d'inscription → collecte des informations → vérification → validation administrative → compte professionnel activé → gestion autonome des informations et du planning → confirmations → surveillance continue par l'administration.

1.6.5 Programmation des gardes

Source du planning → import ou saisie → contrôle des doublons et conflits → validation administrative → publication → notification aux pharmacies concernées → disponibilité dans l'application.

1.7 Règles métier essentielles

- Une pharmacie n'est affichée comme garde active que si elle correspond à un `DutySchedule` valide pour la date/heure courantes.
- Toute confirmation est horodatée (`timestamp` serveur, non modifiable a posteriori).
- L'absence de confirmation n'implique pas la fermeture ; elle dégrade uniquement le niveau de fiabilité affiché.
- Tout signalement passe par un mécanisme de vérification avant impact sur l'affichage public.
- Toute donnée critique (pharmacie, planning) conserve sa source, sa date de dernière modification et son acteur de modification.
- Toute action administrative sensible (validation, suspension, modération) est journalisée dans `AuditLog` .
- Si plusieurs signalements convergents visent la même pharmacie sur une même garde, le statut affiché bascule automatiquement en « à vérifier » jusqu'à contrôle (voir §1.9 Workflow d'incident critique dans le document Workflow).

1.8 Modèle de données et livrables

1.8.1 Entités principales

Entité	Champs indicatifs	Rôle
<code>User</code>	id, nom, email/téléphone, rôle, statut, dates	Compte utilisateur générique (visiteur inscrit, pharmacie, admin, super admin).
<code>Pharmacy</code>	id, nom, adresse, ville, quartier, latitude, longitude, téléphone, statut, source	Fiche pharmacie et localisation géospatiale.

Entité	Champs indicatifs	Rôle
DutySchedule	id, pharmacy_id, date, début, fin, statut, source	Planning de garde d'une pharmacie.
Confirmation	id, pharmacy_id, utilisateur, timestamp, position éventuelle, résultat	Confirmation horodatée d'une garde en cours.
Report	id, pharmacy_id, auteur, type, description, date, statut	Signalement d'anomalie.
AuditLog	id, acteur, action, ressource, timestamp, métadonnées	Journal des actions sensibles.

1.8.2 Règles d'intégrité

Un `DutySchedule` appartient à une seule `Pharmacy`. Une `Confirmation` référence un `DutySchedule` et, le cas échéant, l'utilisateur pharmacie qui l'a émise. Un `Report` référence une `Pharmacy` et, optionnellement, un `DutySchedule` précis. Deux `DutySchedule` d'une même pharmacie ne doivent pas se chevaucher sur une même plage horaire (contrainte de non-chevauchement contrôlée par le service métier, pas seulement par le frontend).

1.9 Architecture applicative et API cible

1.9.1 Stack technique

Note de cadrage. Le cahier des charges initial (v1.0) proposait FastAPI/Python pour le backend. Cette révision remplace ce choix par `Node.js`, afin d'aligner l'équipe backend sur une stack unique en TypeScript avec le frontend. La structure ci-dessous (contrôleurs /

services / repositories) reste transposable telle quelle si le choix final se porte sur Express plutôt que NestJS.

Couche	Technologie	Responsabilité
Frontend web	React + TypeScript + Tailwind CSS	Navigation, carte, recherche, espaces utilisateur/pharmacie/admin.
Mobile (post-MVP)	React Native / Expo, ou PWA pour le MVP	Extension mobile native après validation du pilote.
Backend	Node.js (NestJS ou Express) + TypeScript	Routage HTTP, contrôle d'accès, orchestration des services métier.
ORM / Persistance	Prisma (ou Sequelize/TypeORM)	Mapping objet-relationnel, migrations, transactions.
Base de données	PostgreSQL + PostGIS	Stockage relationnel et requêtes géospatiales (proximité).
Cache / tâches asynchrones	Redis + worker (BullMQ)	Rappels de confirmation, notifications différées.
Authentification	JWT (jsonwebtoken + bcrypt)	Identification, rôles, permissions.
Cartographie	Fournisseur externe (étude coûts/licences)	Géocodage, itinéraire.
Déploiement	Docker, HTTPS, CI/CD	Sauvegardes, monitoring.

1.9.2 Organisation backend recommandée

```
backend/src/  
├─ main.ts
```

```

├── core/
│   ├── config.ts
│   ├── database.ts
│   └── errors.ts
├── modules/
│   ├── users/
│   ├── pharmacies/
│   ├── duty-schedules/
│   ├── confirmations/
│   ├── reports/
│   └── audit/
├── services/
│   ├── search.service.ts      (recherche géospatiale +
proximité)
│   ├── duty-schedule.service.ts (validation, conflits,
publication)
│   ├── confirmation.service.ts (horodatage, fraîcheur,
rappels)
│   ├── report.service.ts      (classification,
escalade, incident critique)
│   └── audit.service.ts
├── controllers/
│   ├── auth.controller.ts
│   ├── pharmacies.controller.ts
│   ├── duty-schedules.controller.ts
│   ├── confirmations.controller.ts
│   ├── reports.controller.ts
│   └── admin.controller.ts
└── security/
    ├── jwt.strategy.ts
    ├── roles.guard.ts
    └── password.ts

```

Le contrôleur ne contient aucune règle métier : il valide la requête, délègue au service, retourne la réponse. Le service applique les règles (conflit de planning, seuil d'incident critique, fraîcheur de confirmation) et écrit l'audit lorsque l'action est sensible — même séparation contrôleur/service que la séparation CRUD/services métier de POSTrack.

1.9.3 API REST — ressources principales

Méthode	Endpoint	Usage
GET	<code>/api/pharmacies/search?city=&lat=&lng=&radius=</code>	Recherche géospatiale des pharmacies de garde valides.
GET	<code>/api/pharmacies/{id}</code>	Fiche pharmacie complète (planning, dernière confirmation, source).
PATCH	<code>/api/pharmacies/{id}</code>	Mise à jour des informations par la pharmacie titulaire ou l'admin.
GET	<code>/api/pharmacies/{id}/duty-schedules</code>	Planning de garde d'une pharmacie.
POST	<code>/api/pharmacies/{id}/duty-schedules</code>	Programmation d'une garde (admin) ou import.
POST	<code>/api/duty-schedules/{id}/confirmations</code>	Confirmation de garde par la pharmacie.
GET	<code>/api/duty-schedules/{id}/confirmations</code>	Historique des confirmations d'une garde.

Méthode	Endpoint	Usage
POST	<code>/api/reports</code>	Création d'un signalement (utilisateur).
PATCH	<code>/api/reports/{id}</code>	Traitement d'un signalement (pharmacie ou admin).
GET	<code>/api/admin/reports?status=</code>	Liste des signalements pour modération.
POST	<code>/api/admin/pharmacies/{id}/validate</code>	Validation d'un compte pharmacie.
GET	<code>/api/admin/audit-logs</code>	Consultation du journal d'audit.
POST	<code>/api/auth/login</code> / <code>/api/auth/refresh</code>	Authentification.

Tous les endpoints d'écriture contrôlent le JWT, le rôle et, pour les ressources pharmacie, l'appartenance de la ressource au titulaire ou à l'administrateur de zone — le serveur ne doit jamais se fier à un contrôle réalisé uniquement côté frontend.

1.10 Sécurité et confidentialité

HTTPS partout ; mots de passe hachés (bcrypt) ; JWT avec expiration et rotation ; contrôle d'accès par rôles ; rate limiting et validation stricte des entrées ; protection contre injections, XSS, CSRF ; journalisation des actions sensibles ; collecte minimale de la position GPS avec

transparence sur son usage ; sauvegardes chiffrées et stratégie de restauration testée.

1.11 Exigences non fonctionnelles

Catégorie	Exigence
Performance	Temps de réponse API inférieur à 500 ms au p95 sur une recherche géospatiale usuelle, pour un jeu de test représentatif d'une ville pilote.
Sécurité	JWT obligatoire sur les routes protégées ; rôle et appartenance de ressource vérifiés côté serveur.
Fiabilité	Une confirmation ou une programmation de garde en échec ne doit jamais laisser un statut « garde active » incohérent affiché publiquement.
Traçabilité	Confirmations, validations, signalements et suspensions doivent être auditable.
Ergonomie	Le statut de fiabilité (vérifiée, confirmée récemment, ancienne, signalée) doit être visible en un coup d'œil sur chaque fiche.
Portabilité	Le déploiement doit rester compatible avec Docker et une stack Node.js/PostgreSQL standard, répliquable pour une nouvelle ville sans refonte.

1.12 Plan de tests et critères de recette

Scénario	Résultat attendu
Recherche par ville	Liste des pharmacies de garde valides, triée par proximité.

Scénario	Résultat attendu
Recherche par position	Résultats géospatiaux corrects (PostGIS), triés par distance réelle.
Confirmation de garde	Statut publié avec horodatage exact ; compteur de fraîcheur mis à jour.
Absence de confirmation	La pharmacie reste visible avec un statut dégradé, pas de fermeture automatique.
Signalement	Ticket créé, notifié à la pharmacie, traçable jusqu'à clôture.
Signalements multiples convergents	Bascule automatique en statut « à vérifier » (voir workflow d'incident critique).
Validation pharmacie	Compte activé uniquement après validation admin.
Accès non autorisé	L'API refuse toute écriture hors périmètre du rôle ou de la ressource.
Sauvegarde / restauration	Les données sont sauvegardées et restaurables sans perte.

La recette est prononcée lorsque toutes les fonctionnalités du MVP (§1.13) sont opérationnelles, que le parcours utilisateur complet (recherche → fiche → action) fonctionne sans anomalie bloquante, et que le cycle planning → confirmation → publication est démontré sur la ville pilote.

1.13 MVP exploitable — ville pilote

Une première ville pilote ; catalogue de pharmacies vérifiées ; planning de garde ; recherche et carte ; position et tri par distance ; fiche pharmacie + appel + itinéraire ; comptes pharmacies ; confirmation de

garde ; signalement ; back-office de validation ; logs, sauvegardes et sécurité de base.

1.14 Hors périmètre du MVP

Recherche de disponibilité de produits/stocks ; favoris et notifications personnalisées avancées ; historique et analytics étendus ; API partenaire ; intégration avec d'autres services de santé ; support multilingue ; application mobile native (le MVP reste web responsive / PWA) ; automatisation complète de l'import des plannings (import/saisie assistée uniquement au MVP).

1.15 Glossaire

Terme	Définition
Pharmacie de garde	Pharmacie assurant un service continu (nuit, dimanche, jour férié) selon un planning programmé.
DutySchedule	Créneau de garde programmé pour une pharmacie, avec date de début et de fin.
Confirmation	Déclaration horodatée par la pharmacie qu'elle assure effectivement sa garde en cours.
Fraîcheur	Ancienneté de la dernière confirmation ou mise à jour, utilisée pour qualifier la fiabilité affichée.
Report (Signalement)	Alerte émise par un utilisateur sur une anomalie constatée (fermeture, information erronée).
Incident critique	Situation où plusieurs signalements convergents remettent en cause la fiabilité d'une garde affichée.

Terme	Définition
Back-office	Interface d'administration de validation, modération et supervision.
AuditLog	Journal des actions sensibles (validation, modération, suspension).
MVP	Périmètre minimal exploitable, limité à la ville pilote.

1.16 Conclusion et critères de réception

Indicateur	Cible
Parcours de recherche utilisateur	100 % opérationnel en recette (ville pilote)
Workflow de confirmation de garde	Horodatage et publication fonctionnels
Workflow de signalement	Traçable de la création à la clôture
Back-office de validation	Opérationnel pour pharmacies, gardes et signalements
Sécurité et accès par rôle	4 rôles implémentés et testés
Couverture de tests backend	Au moins 70 % sur les modules critiques (recherche, confirmation, signalement)
Temps de réponse API	< 500 ms au p95 sur la recherche géospatiale

La solution sera considérée comme conforme lorsque le parcours *Recherche → Fiche pharmacie → Action* est démontré sur la ville pilote,

que la confirmation de garde publie un statut horodaté fiable, que le back-office permet de valider une pharmacie et de traiter un signalement de bout en bout, et que les exigences de sécurité et de sauvegarde sont vérifiées.

Fin du cahier des charges — PharmaGarde CM v2.0